Table des matières

Labo 05 — Diagnostiquer, accélérer, transformer 1
Prérequis 1
Étape 0 — Poser le contexte 1
Étape 1 — Le cache de résultats 2
Étape 2 — Le cache de métadonnées 2
Étape 3 — Provoquer (et lire) un spilling 2
Étape 4 — Où observer : temps réel vs historique 3
Étape 5 — Transformer : fenêtrage et semi-structuré 4
Étape 6 *(aperçu)* — Une vue matérialisée [Enterprise+] 4
Nettoyage 5
Récapitulatif 5

Labo 05 — Diagnostiquer, accélérer, transformer

Module 5/8 — Formation Snowflake (préparation SnowPro Core, COF-C03)

But du labo — *Voir* le domaine 4 : les trois caches en action, un spilling dans un Query Profile, les vues d'observabilité (`INFORMATION_SCHEMA` vs `ACCOUNT_USAGE` + le profil en SQL), puis transformer avec le fenêtrage et `FLATTEN`. Comptez 30 à 40 minutes. Tout se fait dans un worksheet Snowsight.

Prérequis

- Le Cours 00 est fait : entrepôt `SNOWPRO_WH` + base `SNOWPRO_DB.SANDBOX`.
- Édition. L'étape 6 (vue matérialisée) exige Enterprise+, l'édition que le Labo 00 fait prendre à l'inscription. Si vous avez pris Standard, elle sera à lire.

Étape 0 — Poser le contexte

```
USE ROLE SYSADMIN;
USE WAREHOUSE SNOWPRO_WH;
USE DATABASE SNOWPRO_DB;
USE SCHEMA SANDBOX;
```


© 2026 iAlexMG. Tous droits réservés.

Étape 1 — Le cache de résultats

Théorie : `Theorie_05_SNOW.md`, section 3.

```
-- 1re exécution : l'entrepôt calcule.
SELECT L_RETURNFLAG, COUNT(*) AS n, AVG(L_QUANTITY) AS q
FROM   SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.LINEITEM
GROUP  BY L_RETURNFLAG;

-- Relancez À L'IDENTIQUE (recollez la requête ci-dessus, ou Ctrl+Entrée dessus) :
-- réponse quasi instantanée -> cache de résultats.

-- On DÉSACTIVE le cache et on relance : Snowflake RECALCULE.
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
SELECT L_RETURNFLAG, COUNT(*) AS n, AVG(L_QUANTITY) AS q
FROM   SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.LINEITEM
GROUP  BY L_RETURNFLAG;
ALTER SESSION SET USE_CACHED_RESULT = TRUE;    -- on remet
```

Ce que vous observez. La 2e exécution (cache actif) revient en un éclair ; une fois `USE_CACHED_RESULT = FALSE`, la requête recalcule (durée à nouveau visible). Le cache de résultats ressert une requête identique tant que les données ne changent pas.

Étape 2 — Le cache de métadonnées

Théorie : `Theorie_05_SNOW.md`, section 3.

```
-- COUNT(*) PUR : répondu par les MÉTADONNÉES, sans scan (quasi instantané).
SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.LINEITEM;

-- COUNT FILTRÉ : là, il faut LIRE pour compter dans la plage.
SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.LINEITEM
WHERE  L_QUANTITY > 40;
```

Ce que vous observez. Le `COUNT(*)` pur répond instantanément (le total vit dans les métadonnées des micro-partitions) ; le `COUNT` filtré, lui, doit scanner. Comparez les durées dans Query History.

Étape 3 — Provoquer (et lire) un spilling

Théorie : `Theorie_05_SNOW.md`, section 2.

```
-- Un gros GROUP BY à haute cardinalité, sur un PETIT entrepôt (XS) : la mémoire
-- déborde -> SPILLING. On désactive le cache pour forcer un vrai calcul.
```


© 2026 iAlexMG. Tous droits réservés.

```sql
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
SELECT L_ORDERKEY, SUM(L_EXTENDEDPRICE) AS total
FROM   SNOWFLAKE_SAMPLE_DATA.TPCH_SF10.LINEITEM   -- 60 M lignes
GROUP  BY L_ORDERKEY
ORDER  BY total DESC
LIMIT  100;
ALTER SESSION SET USE_CACHED_RESULT = TRUE;
```

Puis : Query History → cette requête → Query Profile. Cherchez, sur le nœud d'agrégation, « Bytes spilled to local storage » (voire *remote*).

Ce que vous observez. Le profil affiche des octets déversés sur disque : l'entrepôt `XS` a manqué de mémoire. Le remède direct = grossir l'entrepôt. *(Si vous ne voyez aucun spill, la requête n'a pas assez débordé : remplacez `TPCH_SF10` par `TPCH_SF100` pour forcer le trait.)*

Étape 4 — Où observer : temps réel vs historique

Théorie : `Theorie_05_SNOW.md`, section 5.

```sql
-- TEMPS RÉEL, par base : les dernières requêtes (colonnes toujours présentes ici).
SELECT QUERY_ID, LEFT(QUERY_TEXT, 40) AS requete, WAREHOUSE_NAME,
       BYTES_SCANNED, COMPILATION_TIME, EXECUTION_TIME, TOTAL_ELAPSED_TIME
FROM   TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())
ORDER  BY START_TIME DESC
LIMIT  10;

-- Le Query Profile EN SQL : d'abord une requête qui scanne une table...
SELECT COUNT(*) FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.LINEITEM WHERE L_SHIPDATE = '1995-06-15';
-- ...puis on lit son pruning tout de suite (GET_QUERY_OPERATOR_STATS = le profil en table).
SELECT operator_type,
       operator_statistics:pruning:partitions_scanned::int AS scanned,
       operator_statistics:pruning:partitions_total::int   AS total
FROM   TABLE(GET_QUERY_OPERATOR_STATS(LAST_QUERY_ID()))
WHERE  operator_type = 'TableScan';

-- HISTORIQUE LONG (1 an) mais DIFFÉRÉ + privilégié : ACCOUNT_USAGE (base SNOWFLAKE).
USE ROLE ACCOUNTADMIN;
SELECT QUERY_ID, WAREHOUSE_NAME, PARTITIONS_SCANNED,
       BYTES_SPILLED_TO_LOCAL_STORAGE, TOTAL_ELAPSED_TIME
FROM   SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
ORDER  BY START_TIME DESC
LIMIT  10;
USE ROLE SYSADMIN;
```


© 2026 iAlexMG. Tous droits réservés.

Ce que vous observez. `INFORMATION_SCHEMA.QUERY_HISTORY()` répond en temps réel ; `GET_QUERY_OPERATOR_STATS` vous donne le pruning exact ; `ACCOUNT_USAGE.QUERY_HISTORY` offre un an d'historique mais avec une latence, et vos requêtes de ce labo peuvent ne pas encore y figurer. C'est le piège d'examen : *long historique et différé* = `ACCOUNT_USAGE`.

Étape 5 — Transformer : fenêtrage et semi-structuré

Théorie : `Theorie_05_SNOW.md`, section 6.

```
-- a) FENÊTRAGE : un rang PAR région, en gardant chaque ligne de détail.
CREATE OR REPLACE TABLE ventes (vendeur STRING, region STRING, montant NUMBER);
INSERT INTO ventes VALUES
  ('Alix','QC',100), ('Bo','QC',150), ('Cam','ON',120), ('Dee','ON',90);

SELECT vendeur, region, montant,
       RANK() OVER (PARTITION BY region ORDER BY montant DESC) AS rang
FROM   ventes;

-- b) SEMI-STRUCTURÉ : éclater un tableau JSON en une ligne par élément (FLATTEN).
CREATE OR REPLACE TABLE evenements (id INT, charge VARIANT);
INSERT INTO evenements SELECT 1, PARSE_JSON('{"tags":["a","b","c"]}');

SELECT e.id, f.value::string AS tag
FROM   evenements e,
       LATERAL FLATTEN(input => e.charge:tags) f;
```

Ce que vous observez. Le fenêtrage donne à chaque vendeur son rang régional sans replier les lignes (contrairement à `GROUP BY`). `FLATTEN` transforme le tableau `["a","b","c"]` en trois lignes, c'est l'aplatissement JSON.

Étape 6 *(aperçu)* — Une vue matérialisée [Enterprise+]

Théorie : `Theorie_05_SNOW.md`, section 4.

```
-- Stocke un résultat précalculé et le maintient à jour tout seul.
CREATE OR REPLACE MATERIALIZED VIEW mv_ventes_region AS
  SELECT region, SUM(montant) AS total FROM ventes GROUP BY region;

SELECT * FROM mv_ventes_region;
SHOW MATERIALIZED VIEWS;
```

Ce que vous observez. La vue matérialisée stocke l'agrégat (contrairement à une vue standard qui recalcule) ; utile pour une agrégation coûteuse relue souvent. Snowflake la rafraîchit quand la table de base change.

© 2026 iAlexMG. Tous droits réservés.

Nettoyage

```sql
DROP MATERIALIZED VIEW IF EXISTS mv_ventes_region;
DROP TABLE IF EXISTS ventes;
DROP TABLE IF EXISTS evenements;
ALTER SESSION UNSET USE_CACHED_RESULT;   -- remet la valeur par défaut
```

Récapitulatif

- ☐ J'ai vu le cache de résultats (relance instantanée, puis recalcul sans lui).
- ☐ J'ai vu le cache de métadonnées (`COUNT(*)` sans scan).
- ☐ J'ai provoqué et lu un spilling dans un Query Profile.
- ☐ Je distingue `INFORMATION_SCHEMA` (temps réel) de `ACCOUNT_USAGE` (historique différé), et j'ai lu le pruning via `GET_QUERY_OPERATOR_STATS`.
- ☐ J'ai transformé avec le fenêtrage (`RANK() OVER`) et `LATERAL FLATTEN`.
- ☐ J'ai créé une vue matérialisée.

Tout coché, vous tenez le 2e domaine le plus lourd. Cap sur le Cours 06, la collaboration de données (partage, clonage, Time Travel).

© 2026 iAlexMG. Tous droits réservés.